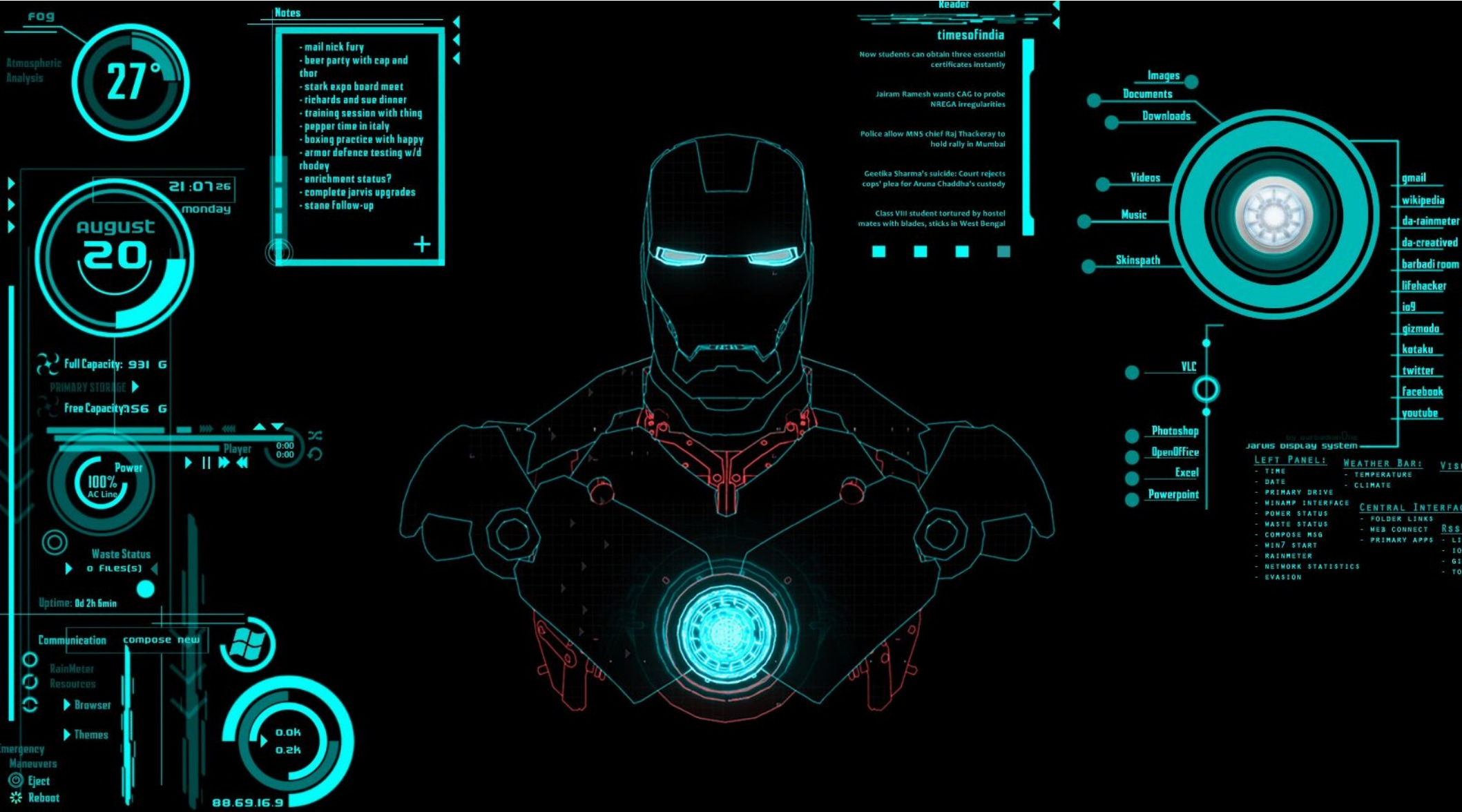




12.OpenGL e Objetos Tridimensionais

Prof. Alexandre Krohn

Roteiro

- GLM
- Sistemas de Coordenadas
- Espaços de Coordenadas
- Projeção Ortográfica
- Projeção em Perspectiva
- Programando em 3D

GLM

- **GLM : OpenGL Mathematics** é uma biblioteca em C++ que implementa os mesmos tipos de dados e as funções matemáticas presentes na GLSL
- Ela pode ser utilizada de forma independente da OpenGL e possui as funções necessárias da álgebra linear para os cálculos das transformações necessárias em 3D
- Usaremos essa função em conjunto com as outras já configuradas

GLM

- GLM implementa:
 - Funções de cálculo vetorial,
 - Operações sobre matrizes,
 - Além dos tipos `vec3`, `vec4` e outros que já vimos quando estudamos shaders

Instalando GLM

- Faça o download da biblioteca a partir de <https://glm.g-truc.net/0.9.9/index.html>

Instalando GLM

OpenGL Mathematics

← → ↺ 🏠 🔒 https://glm.g-truc.net/0.9.9/index.html 📄 ⋮ 🛡️ ☆ 📖 📄 🛑 ⚙️ Se 👤 🗣️ 🏠 🛠️



Download GLM 0.9.9.7
2020-01-05

Manual
API Documentation

Release Notes
Downloads
Report Bug

Stable Branch Snapshot
build passing

Master Branch Snapshot
build passing

Git Repository

Stack Overflow
GLSL Specification
GLSL Man Pages

OpenGL Mathematics

GLSL + Optional features = OpenGL Mathematics (GLM)
A C++ mathematics library for graphics programming

OpenGL Mathematics (GLM) is a header only C++ mathematics library for graphics software based on the [OpenGL Shading Language \(GLSL\)](#) specification. GLM provides classes and functions designed and implemented with the same naming conventions and functionalities than GLSL so that anyone who knows GLSL can use GLM.

This project isn't limited to GLSL features. An extension system, based on the GLSL extension conventions, provides extended capabilities: matrix transformations, random numbers, noise, etc...

This library works perfectly with OpenGL but it also ensures interoperability with other third party libraries and SDK. It is a good candidate for software development in graphics processing, physic simulations and any development context that requires a simple and convenient mathematics library.

GLM is written in C++98 but can take advantage of C++11 when supported by the compiler. It is a platform independent library with no dependencies.

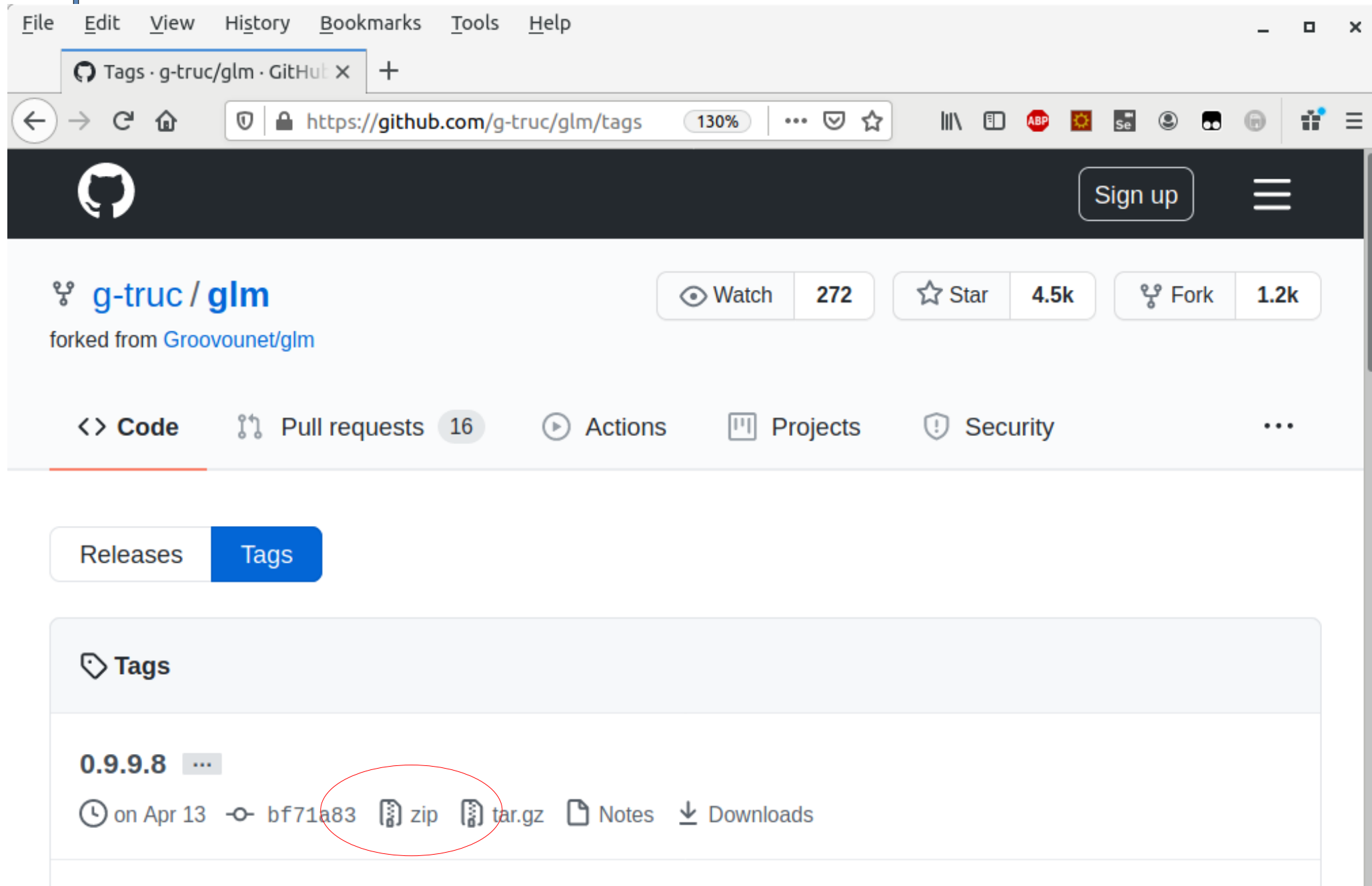
- [Apple Clang](#) 6.0 and higher
- [GCC](#) 4.7 and higher
- [Intel C++ Composer](#) XE 2013 and higher
- [LLVM](#) 3.4 and higher
- [Visual C++](#) 2013 and higher
- [CUDA](#) 9.0 and higher (experimental)
- Any conform C++11 compiler

For more information about GLM, please have a look at [the manual](#) and [the API reference documentation](#).

The source code and the documentation, including this manual, are licensed under [the Happy Bunny License \(Modified MIT\)](#) or the [MIT License](#).

Thanks for contributing to the project by [submitting issues](#) for bug reports and feature requests. Any feedback is welcome at glm@g-truc.net.

Instalando GLM



Instalando GLM

- Descompacte o arquivo de modo que seu computador fique com uma pasta `C:\glm`
- Configure as Build Options do Code::Blocks, dentro da aba Search directories / Compiler
 - Adicione C:\glm

Roteiro

- GLM
- **Sistemas de Coordenadas**
- Espaços de Coordenadas
- Projeção Ortográfica
- Projeção em Perspectiva
- Programando em 3D

Sistemas de Coordenadas

- Na **aula 06**, vimos como usar matrizes de transformação para realizar as operações de translação, rotação e escala nos vértices dos objetos.
- O OpenGL espera que todos os vértices, que desejamos que se tornem visíveis, estejam em coordenadas de dispositivo normalizadas após cada execução do shader de vértice.
- Ou seja, as coordenadas x , y e z de cada vértice devem estar entre $-1,0$ e $1,0$; as coordenadas fora deste intervalo não serão visíveis.

Sistemas de Coordenadas

- O que é feito geralmente é especificar as coordenadas em um intervalo (ou espaço) determinado pelo desenvolvedor, e no vertex shader transforma-se essas coordenadas em coordenadas de dispositivo normalizadas (**NDC - Normalized Device Coordinates**).
- Essas NDC são então fornecidas ao rasterizador para transformá-los em coordenadas / pixels 2D na tela.

Sistemas de Coordenadas

- A transformação de coordenadas em NDC é realizada passo a passo, onde transformam-se os vértices de um objeto em vários sistemas de coordenadas antes de finalmente transformá-los em NDC.
- A vantagem de transformá-los em vários sistemas de coordenadas intermediárias é que **algumas operações / cálculos são mais fáceis em certos sistemas de coordenadas.**
- Há um total de **5 sistemas de coordenadas** diferentes que são importantes para nós:

5 Sistemas:

- **Local space (or Object space)** - Espaço do objeto
- **World space** – Espaço do mundo
- **View space (ou Eye space)** - Espaço de visualização
- **Clip space** - Espaço de recorte
- **Screen space** – Espaço de tela

Cada um deles é um espaço diferente para o qual os vértices serão transformados antes de finalmente terminar como fragmentos.

Adequação dos espaços

- Exemplos:
 - Modificar o **objeto**, faz mais sentido no **espaço local**
 - Já o cálculo de certas operações no objeto com **relação à posição de outros objetos** faz mais sentido em **coordenadas mundiais**
 - e assim por diante...

Como funciona?

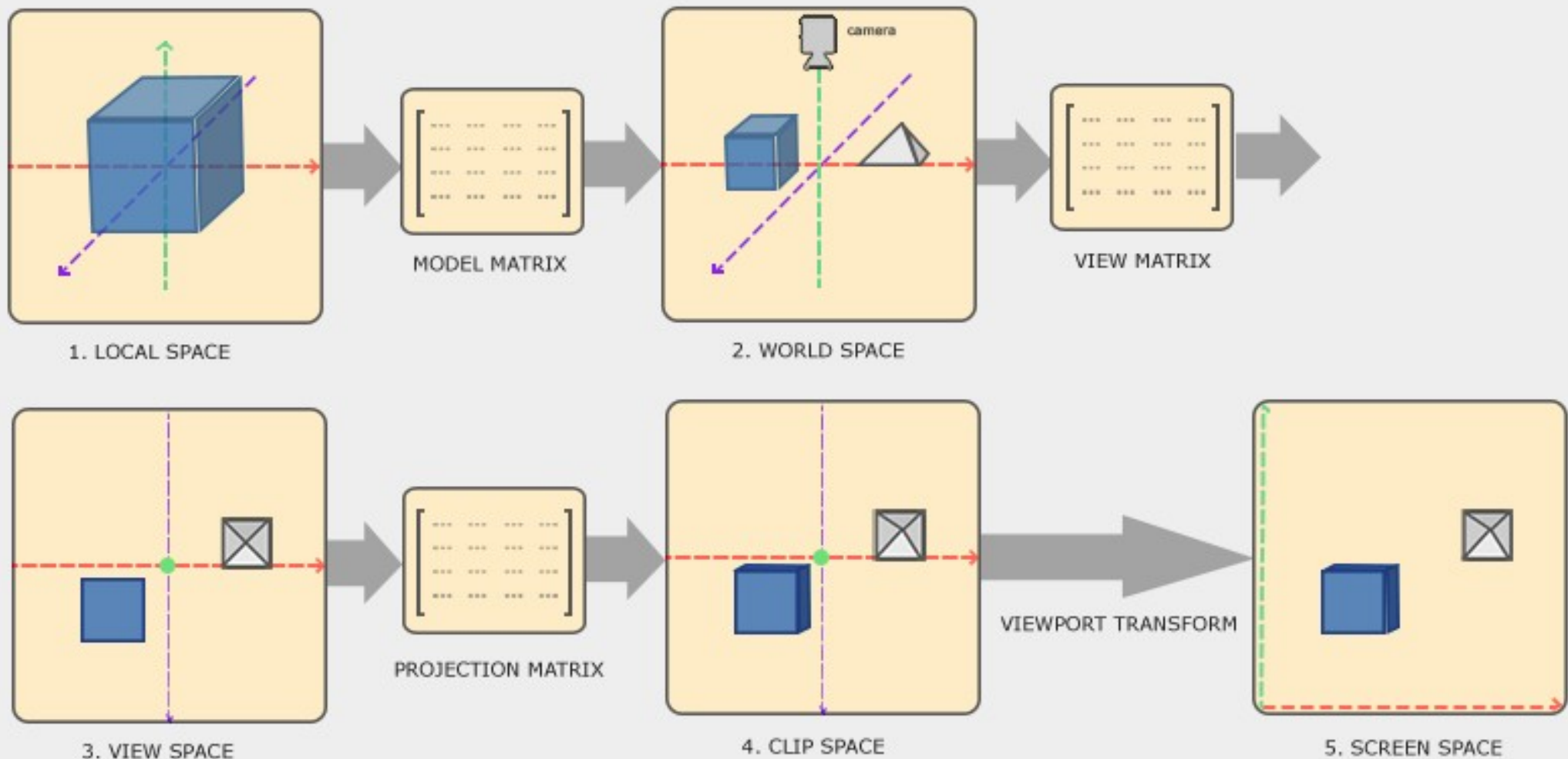
- Para transformar as coordenadas de um espaço para o próximo espaço de coordenadas, usaremos várias matrizes de transformação, das quais as mais importantes são o modelo (**model**), a visualização (**view**) e a matriz de projeção (**projection**).
- As coordenadas de vértice primeiro começam no espaço local como coordenadas locais e são posteriormente processadas em coordenadas mundiais, coordenadas de visualização, coordenadas de recorte e, eventualmente, terminam como coordenadas de tela.
 - **Local → Mundial → Visualização → Recorte → Tela**

Como funciona?

- Para transformar as coordenadas de um espaço para o próximo espaço de coordenadas, usaremos várias matrizes de transformação, das quais as mais importantes são o modelo (**model**), a visualização (**view**) e a matriz de projeção (**projection**).
- As coordenadas de vértice primeiro começam no espaço local como coordenadas locais e são posteriormente processadas em coordenadas mundiais, coordenadas de visualização, coordenadas de recorte e, eventualmente, terminam como coordenadas de tela.

– Local → Mundial → Visualização → Recorte → Tela

Etapas de transformação



Etapas de transformação

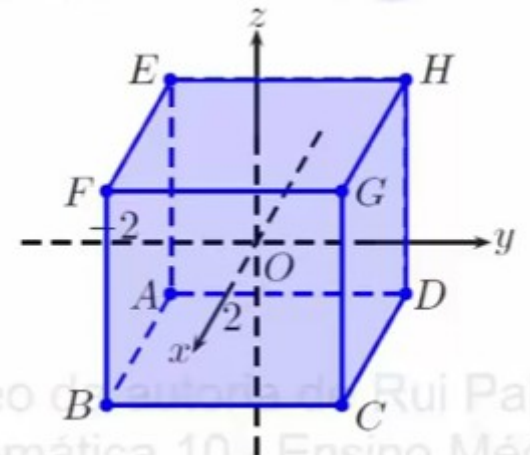
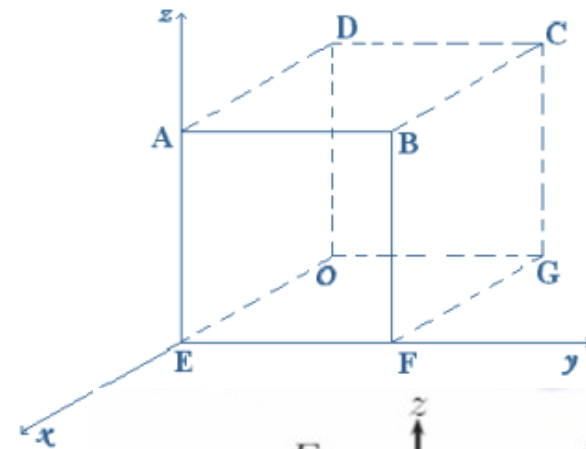
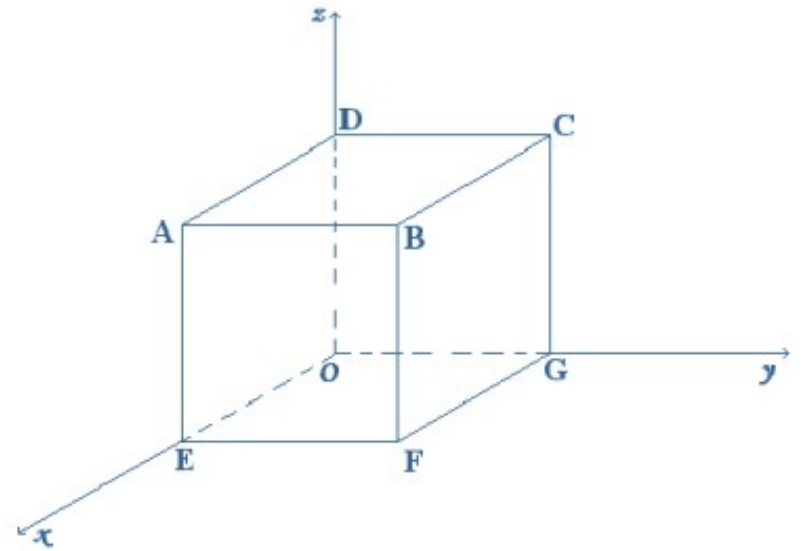
- 1) **Coordenadas locais** são as coordenadas de seu objeto em relação à sua **origem local**; são as coordenadas nas quais seu objeto começa.
- 2) O próximo passo é transformar as coordenadas locais em coordenadas do **espaço mundial**, que são coordenadas em relação a um mundo maior. **Essas coordenadas são relativas a alguma origem global do mundo**, junto com muitos outros objetos também colocados em relação à origem deste mundo.
- 3) Em seguida, transformamos as coordenadas mundiais em **coordenadas do espaço de visualização**, de forma que cada coordenada seja **vista da câmera ou do ponto de vista do observador**.
- 4) Depois que as coordenadas estiverem no espaço de visualização, queremos **projetá-las para recortar as coordenadas**. As coordenadas do clipe são processadas na faixa de -1,0 e 1,0 e determinam quais vértices irão para a tela. A projeção para as coordenadas do espaço do clipe pode adicionar perspectiva se usar a projeção em perspectiva.
- 5) E, por último, transformamos as coordenadas do clipe em **coordenadas de tela** em um processo que chamamos de transformação da janela de visualização que transforma as coordenadas de -1,0 e 1,0 para o intervalo de coordenadas definido por **glViewport**. As coordenadas resultantes são então enviadas ao rasterizador para transformá-las em fragmentos.

Roteiro

- GLM
- Sistemas de Coordenadas
- Espaços de Coordenadas
- Projeção Ortográfica
- Projeção em Perspectiva
- Programando em 3D

Espaço local

- Espaço local é o espaço de coordenadas que é local para o seu objeto, ou seja, onde seu objeto começa.
- Imagine que você criou um cubo em espaço cartesiano. A origem do seu cubo está provavelmente em $(0,0,0)$, embora seu cubo possa terminar em um local diferente em sua aplicação final.



Espaço mundial

- Se puséssemos todos os nossos objetos diretamente em um aplicativo, eles provavelmente começariam em algum lugar posicionados uns dentro dos outros na origem do mundo de $(0,0,0)$,
- Não é isso o que queremos.
- Queremos definir uma posição para cada objeto para posicioná-los dentro de um mundo maior. As coordenadas no espaço do mundo são exatamente o que parecem: as coordenadas de todos os seus vértices relativos a um mundo.
- Este é o espaço de coordenadas onde você deseja que seus objetos sejam transformados de forma que fiquem todos espalhados pelo local (de preferência de forma realista).
- As coordenadas do seu objeto são transformadas do espaço local para o espaço mundial; isso é feito com a matriz do modelo. (**model**)

Espaço mundial : Matriz de modelo

- A **matriz do modelo** é uma matriz de transformação que traduz, dimensiona e / ou gira os objetos para colocá-lo no **mundo** em um local / orientação a que pertencem.
- Pense nisso como transformar uma casa reduzindo-a (era um pouco grande no espaço local), traduzindo-a para as proporções de uma cidade e girando-a um pouco para a esquerda no eixo y para que se encaixe perfeitamente com a casa vizinha.

Espaço de Visualização

- O **espaço de visualização** é o que as pessoas geralmente chamam de **câmera do OpenGL** (às vezes também é conhecido como **espaço da câmera, espaço do olho ou de visão**).
- O espaço de visualização é o resultado da transformação das coordenadas de espaço mundial em coordenadas que estão à frente da visualização do usuário.
- O espaço de visualização é, portanto, o **espaço visto do ponto de vista da câmera**. Isso geralmente é realizado com uma combinação de translações e rotações para transladar / girar a cena de modo que certos itens sejam transformados para a frente da câmera.
- Essas transformações combinadas geralmente são armazenadas dentro de uma **matriz de visualização – view matrix** que transforma as coordenadas mundiais para visualizar o espaço. (Será detalhado mais adiante)

Espaço de Recorte

- No final de cada execução do **vertex shader**, o OpenGL espera que as coordenadas estejam dentro de um intervalo específico e qualquer coordenada que esteja fora desse intervalo é cortada.
- As coordenadas que são cortadas são descartadas, portanto, as coordenadas restantes acabarão como fragmentos visíveis na tela. É também daí que vem o nome do **espaço de recorte** (clipping).
- Como especificar todas as coordenadas visíveis dentro do intervalo **-1.0** e **1.0** não é realmente intuitivo, especificamos nosso próprio conjunto de coordenadas para trabalhar e convertê-las de volta para **NDC** conforme o OpenGL espera.

Espaço de Recorte

- Para transformar as coordenadas de visualização do vértice para o espaço de recorte, definimos a chamada **matriz de projeção**, que especifica um intervalo de coordenadas, por exemplo, **-1000** e **1000** em cada dimensão.
- A matriz de projeção então transforma as coordenadas dentro deste intervalo especificado em coordenadas normalizadas do dispositivo (**-1.0**, **1.0**).
- Todas as coordenadas fora deste intervalo não serão mapeadas entre **-1,0** e **1,0** e, portanto, serão cortadas.
- Com este intervalo que especificamos na matriz de projeção, uma coordenada de (**1250**, **500**, **750**) não seria visível (**1250 > 1000**), uma vez que a coordenada x está fora do intervalo e, portanto, é convertida para uma coordenada superior a 1,0 em NDC e, portanto, é cortada.

Espaço de Recorte

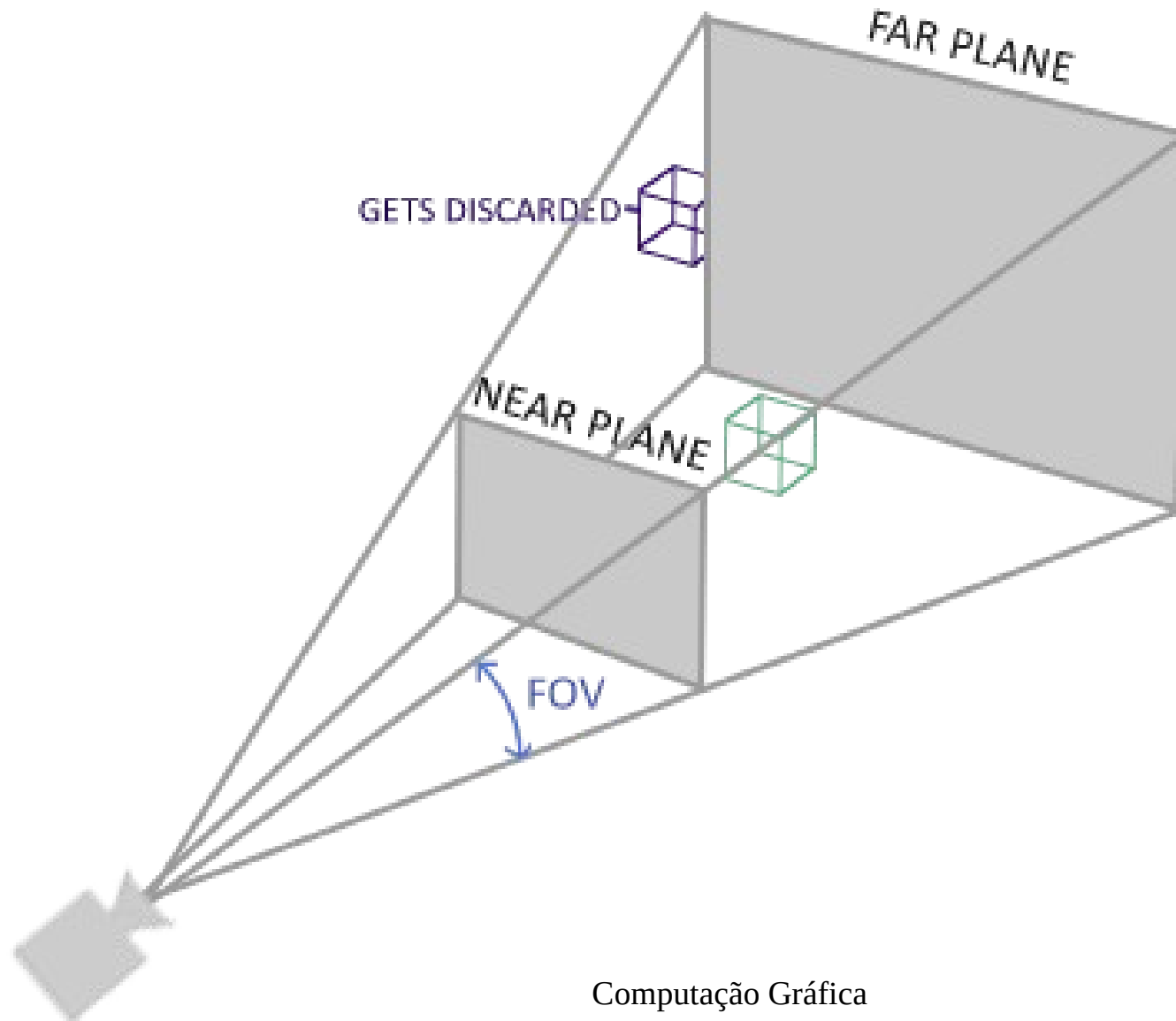
- Para transformar as coordenadas de visualização do vértice para o espaço de recorte, definimos a chamada **matriz de projeção**, que especifica um intervalo de coordenadas, por exemplo, **-1000** e **1000** em cada dimensão.
- A matriz de projeção então transforma as coordenadas dentro deste intervalo especificado em coordenadas normalizadas do dispositivo (**-1.0**, **1.0**).
- Todas as coordenadas fora deste intervalo não serão mapeadas entre **-1,0** e **1,0** e, portanto, serão cortadas.
- Com este intervalo que especificamos na matriz de projeção, uma coordenada de (**1250**, **500**, **750**) não seria visível (**1250 > 1000**), uma vez que a coordenada x está fora do intervalo e, portanto, é convertida para uma coordenada superior a 1,0 em NDC e, portanto, é cortada.

Se uma parte da primitiva (Por exemplo um triângulo – GL_TRIANGLE) é cortada, o OpenGL irá reconstruir o triângulo como um ou mais triângulos para exibir a parte do objeto que “cabe” dentro da área de recorte.

Frustum (tronco)

- Essa caixa de visualização que uma matriz de projeção cria é chamada de **tronco** e cada coordenada que acaba dentro desse tronco vai parar na tela do usuário.
- O processo total para converter coordenadas dentro de um intervalo especificado para **NDC** que pode ser mapeado em coordenadas de espaço de visualização 2D é chamado de projeção
- **A matriz de projeção projeta coordenadas 3D para as coordenadas de dispositivo normalizadas**, que são mais fáceis de mapear para 2D.

Frustum (tronco)



Divisão de Perspectiva

- Depois que todos os vértices são transformados para recortar o espaço, uma operação final chamada **divisão de perspectiva** é realizada, onde dividimos os componentes **x**, **y** e **z** dos vetores de posição pelo componente **w** homogêneo do vetor;
- A divisão de perspectiva é o que transforma as coordenadas do espaço do clipe 4D em coordenadas do dispositivo 3D normalizado. Esta etapa é executada automaticamente no final da etapa do **vertex shader**

glViewport

- É após este estágio que as coordenadas resultantes são mapeadas para as coordenadas da tela (usando as configurações de **glViewport**) e transformadas em fragmentos.
- A **matriz de projeção** para transformar as coordenadas de vista em coordenadas de recorte geralmente assume duas formas diferentes, onde cada forma define seu próprio tronco único.
- Podemos criar uma matriz de **projeção ortográfica** ou uma matriz de **projeção em perspectiva**.

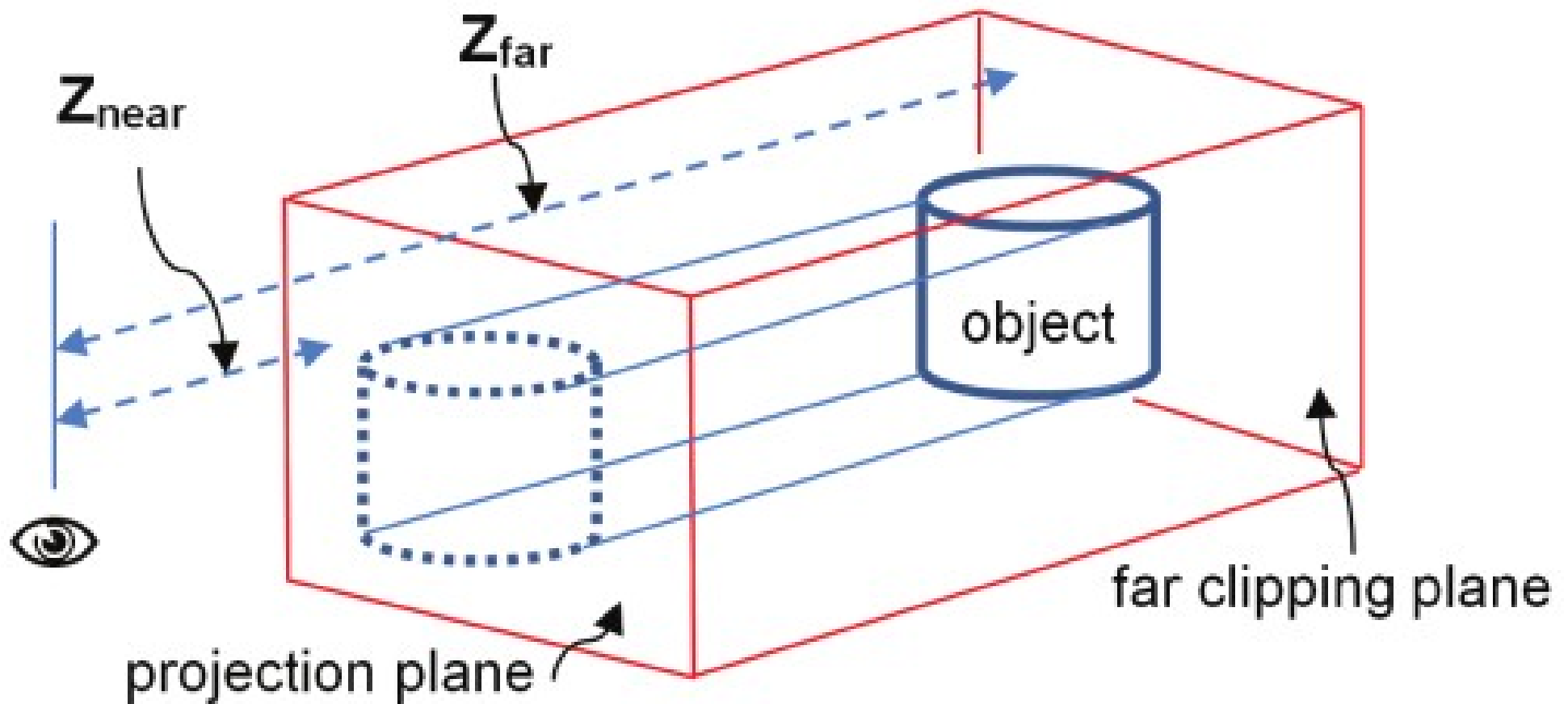
Roteiro

- GLM
- Sistemas de Coordenadas
- Espaços de Coordenadas
- **Projeção Ortográfica**
- Projeção em Perspectiva
- Programando em 3D

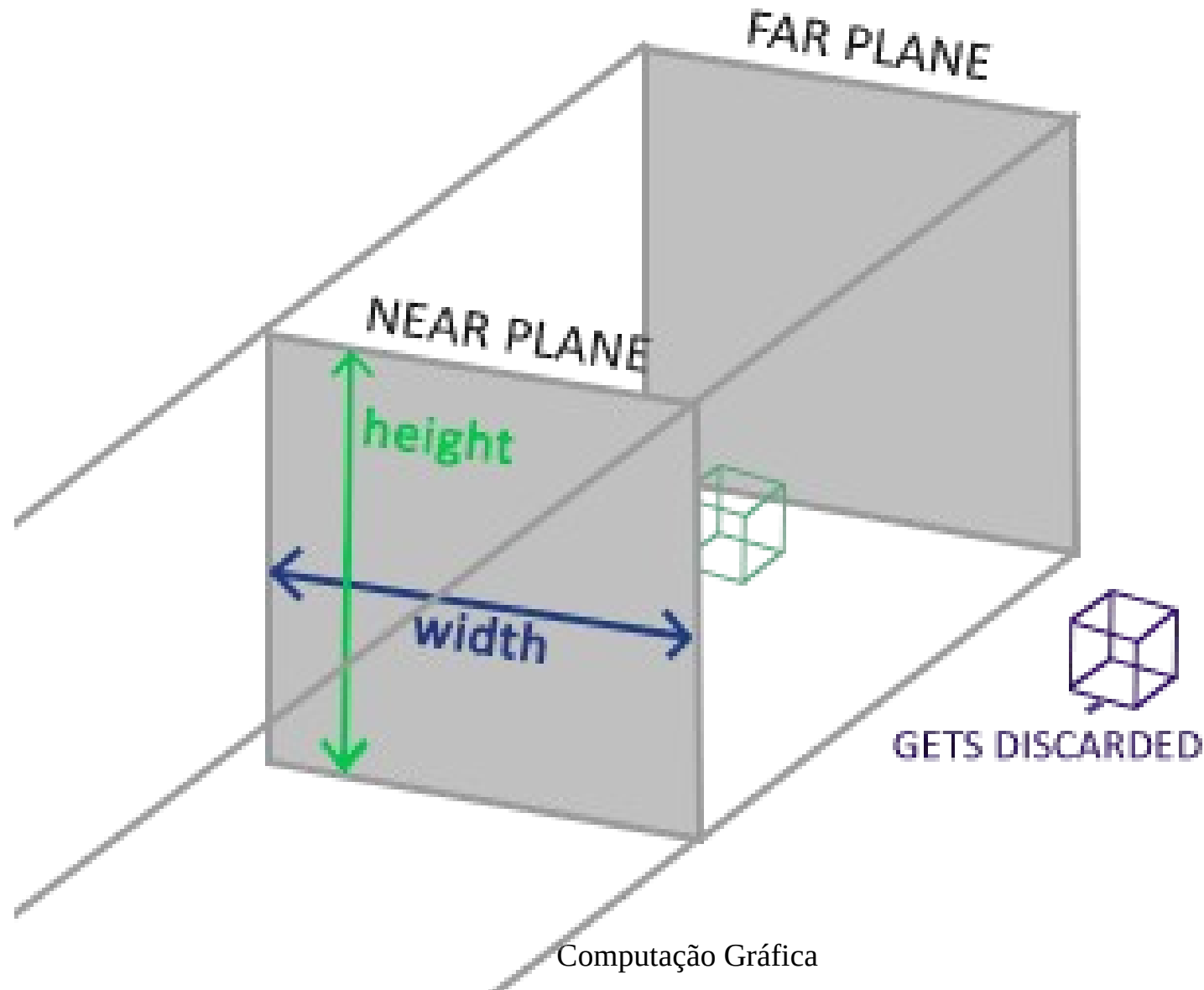
Projeção Ortográfica

- Uma matriz de **projeção ortográfica** define uma caixa de tronco semelhante a um cubo que **define o espaço de recorte onde cada vértice fora desta caixa é recortado**.
- Ao criar uma matriz de projeção ortográfica, especificamos a largura, altura e comprimento do tronco visível.
- Todas as coordenadas dentro deste tronco vão acabar dentro do intervalo NDC após serem transformadas por sua matriz e, portanto, não serão cortadas.

Projeção Ortográfica



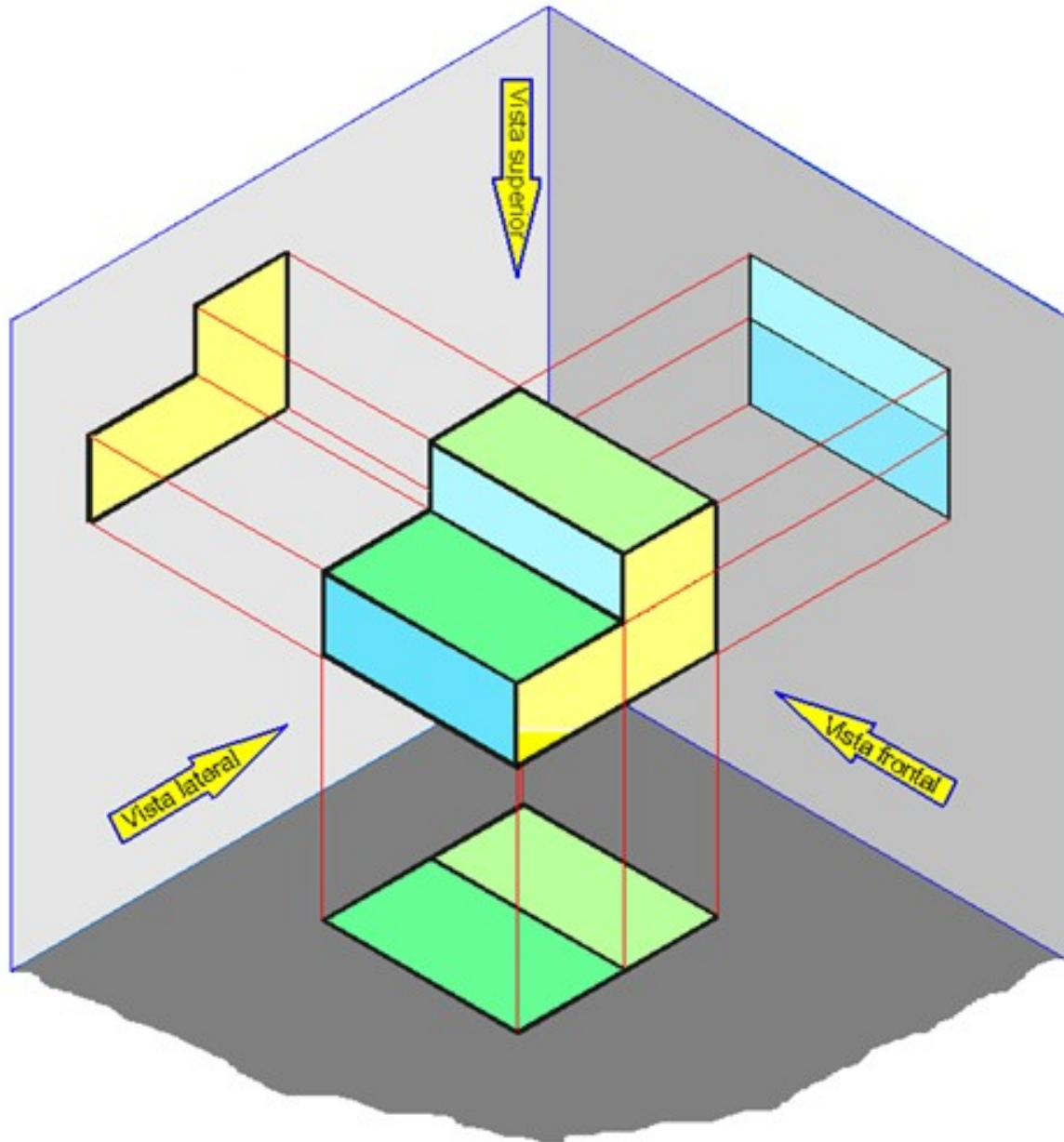
Projeção Ortográfica



Projeção Ortográfica

- O **tronco** define as coordenadas visíveis e é especificado por uma **largura**, uma **altura** e um **plano próximo** e **distante**.
- Qualquer coordenada na frente do plano próximo é recortada e o mesmo se aplica às coordenadas atrás do plano distante.
- O **tronco ortográfico mapeia diretamente** todas as coordenadas dentro do tronco para coordenadas de dispositivo normalizadas sem quaisquer efeitos colaterais especiais, uma vez que não tocará o componente w do vetor transformado; se o componente w permanecer igual a 1,0, a divisão da perspectiva não mudará as coordenadas.

Projeção Ortográfica



Também conhecida como projeção ortogonal, muito utilizada na engenharia

Matriz de Projeção Ortográfica

- Para criar uma **matriz de projeção ortográfica**, usamos a função embutida do GLM **`glm::ortho`**:

```
glm::ortho(0.0f, 800.0f, 0.0f, 600.0f, 0.1f, 100.0f);
```

- Os primeiros dois parâmetros especificam as coordenadas esquerda e direita do tronco e o terceiro e quarto parâmetros especificam as partes inferior e superior do tronco.
- Com esses 4 pontos, definimos o tamanho dos planos próximos e distantes e os parâmetros 5 e 6 definimos as distâncias entre os planos próximos e distantes. **Esta matriz de projeção específica transforma todas as coordenadas entre esses valores de intervalo x, y e z em coordenadas de dispositivo normalizadas.**

Roteiro

- GLM
- Sistemas de Coordenadas
- Espaços de Coordenadas
- Projeção Ortográfica
- **Projeção em Perspectiva**
- Programando em 3D

Projeção em Perspectiva



Projeção em Perspectiva

- No mundo real, objetos **mais próximos** parecem ser muito **maiores** do que objetos distantes
- Essa sensação é causada pelo ponto de vista, ou perspectiva que temos do objeto.
- A projeção em perspectiva gera resultados muito mais próximos da realidade, e portanto mais “aceitáveis” para os observadores da imagem.

Matriz de Projeção de Perspectiva



- Devido à perspectiva, as linhas parecem coincidir a uma determinada distância.
- Este é exatamente o efeito que a projeção de perspectiva tenta imitar e o faz usando uma **matriz de projeção de perspectiva**.

Matriz de Projeção de Perspectiva

- A **matriz de projeção** mapeia um determinado intervalo de tronco para recortar o espaço, mas também **manipula o valor w** de cada coordenada de vértice de forma que ***quanto mais longe uma coordenada de vértice estiver do visualizador, maior será o componente w .***
- Uma vez que as coordenadas são transformadas no espaço de recorte, elas ficam no intervalo $-w$ a w (qualquer coisa fora desse intervalo é cortado).
- O OpenGL requer que as coordenadas visíveis fiquem entre o intervalo $-1,0$ e $1,0$ como a saída final do **vertex shader**, portanto, uma vez que as coordenadas estão no espaço do clipe, a divisão da perspectiva é aplicada às coordenadas do espaço do clipe

Matriz de Projeção de Perspectiva

$$out = \begin{pmatrix} x/w \\ y/w \\ z/w \end{pmatrix}$$

- Cada componente da coordenada do vértice é dividido por seu componente w , obtendo valores menores para coordenadas de vértice quanto mais longe um vértice estiver do visualizador.
- Esta é outra razão pela qual o componente w é importante, uma vez que nos ajuda na projeção em perspectiva.
- As coordenadas resultantes estão então no espaço normalizado do dispositivo.

Matriz de Projeção de Perspectiva

- Uma matriz de projeção de perspectiva pode ser criada no GLM da seguinte forma:

```
glm::mat4 proj = glm::perspective(glm::radians(45.0f),  
(float)width/(float)height, 0.1f, 100.0f);
```

- Seu **primeiro parâmetro** define o valor **fov**, que significa campo de visão e define o quão grande é o espaço de visão. Para uma visualização realista, geralmente é definido como **45 graus**, mas para resultados mais no estilo **Doom**, você pode definir um valor mais alto.
- O **segundo parâmetro** define a proporção da imagem que é calculada dividindo a largura da janela de visualização por sua altura.
- O **terceiro e o quarto parâmetros** definem os **planos próximo e distante** do tronco. Normalmente definimos a distância próxima como **0,1** e a distância distante como **100,0**. Todos os vértices entre os planos próximo e distante e dentro do tronco serão renderizados.

Matriz de Projeção de Perspectiva

- Uma matriz de projeção de perspectiva pode ser criada no GLM da seguinte forma:

```
glm::mat4 proj = glm::perspective(glm::radians(45.0f),  
(float)width/(float)height, 0.1f, 100.0f);
```

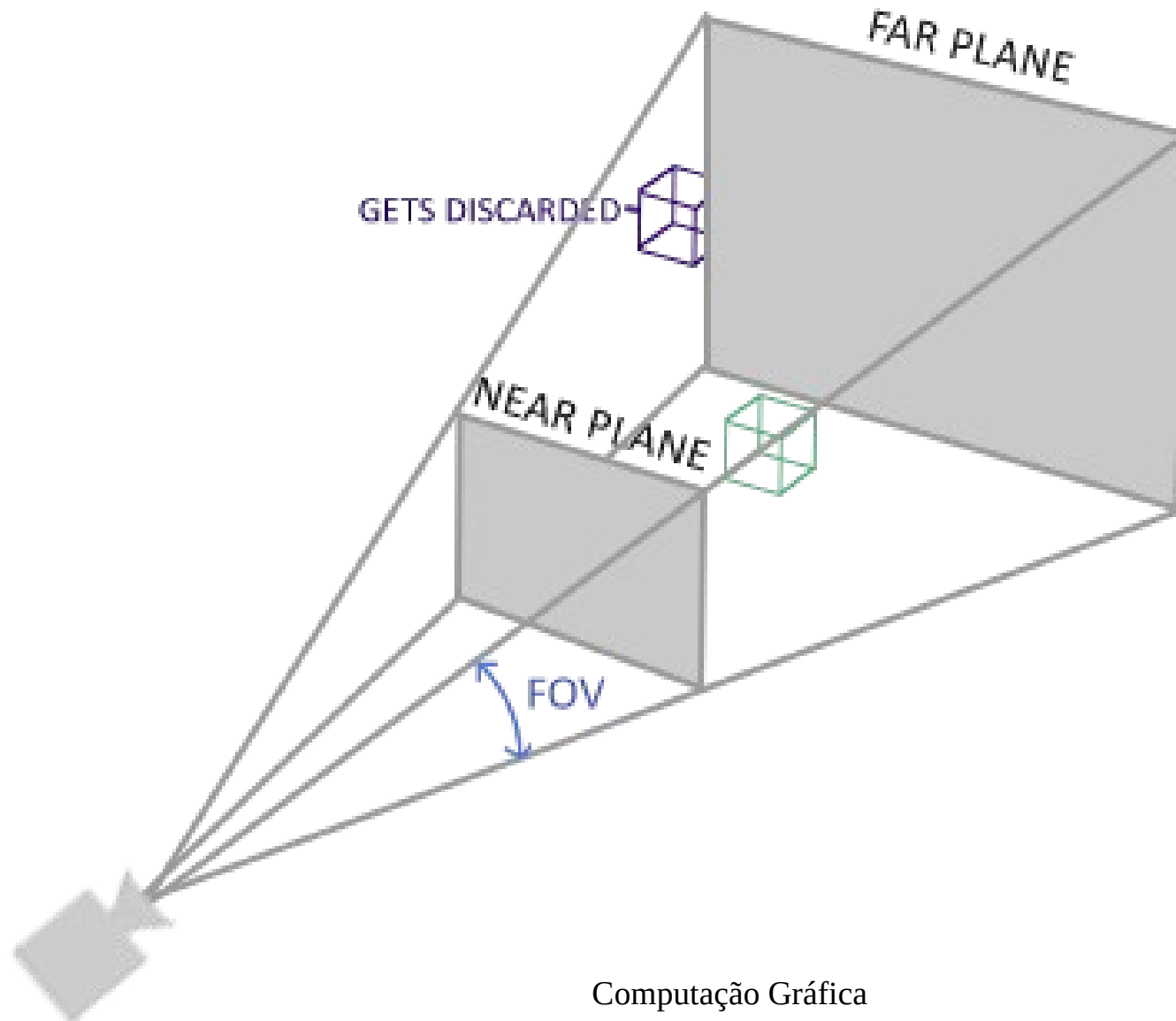
- Seu primeiro parâmetro define o valor **fov**, que significa campo de visão e define o quão grande é o espaço de visão. Para uma visualização realista, geralmente é definido como **45 graus**, mas para resultados mais no estilo **Doom**, você pode definir um valor mais alto.
- O segundo parâmetro define a proporção da imagem que é calculada dividindo a largura da janela de visualização por sua altura.

Sempre que o valor próximo de sua matriz de perspectiva é **definido muito alto** (como **10.0**), o OpenGL irá cortar todas as coordenadas perto da câmera (**entre 0,0 e 10,0**), o que pode gerar um resultado visual que você talvez tenha visto antes em videogames onde você pode **ver através de certos objetos** ou paredes ao se mover muito perto deles.

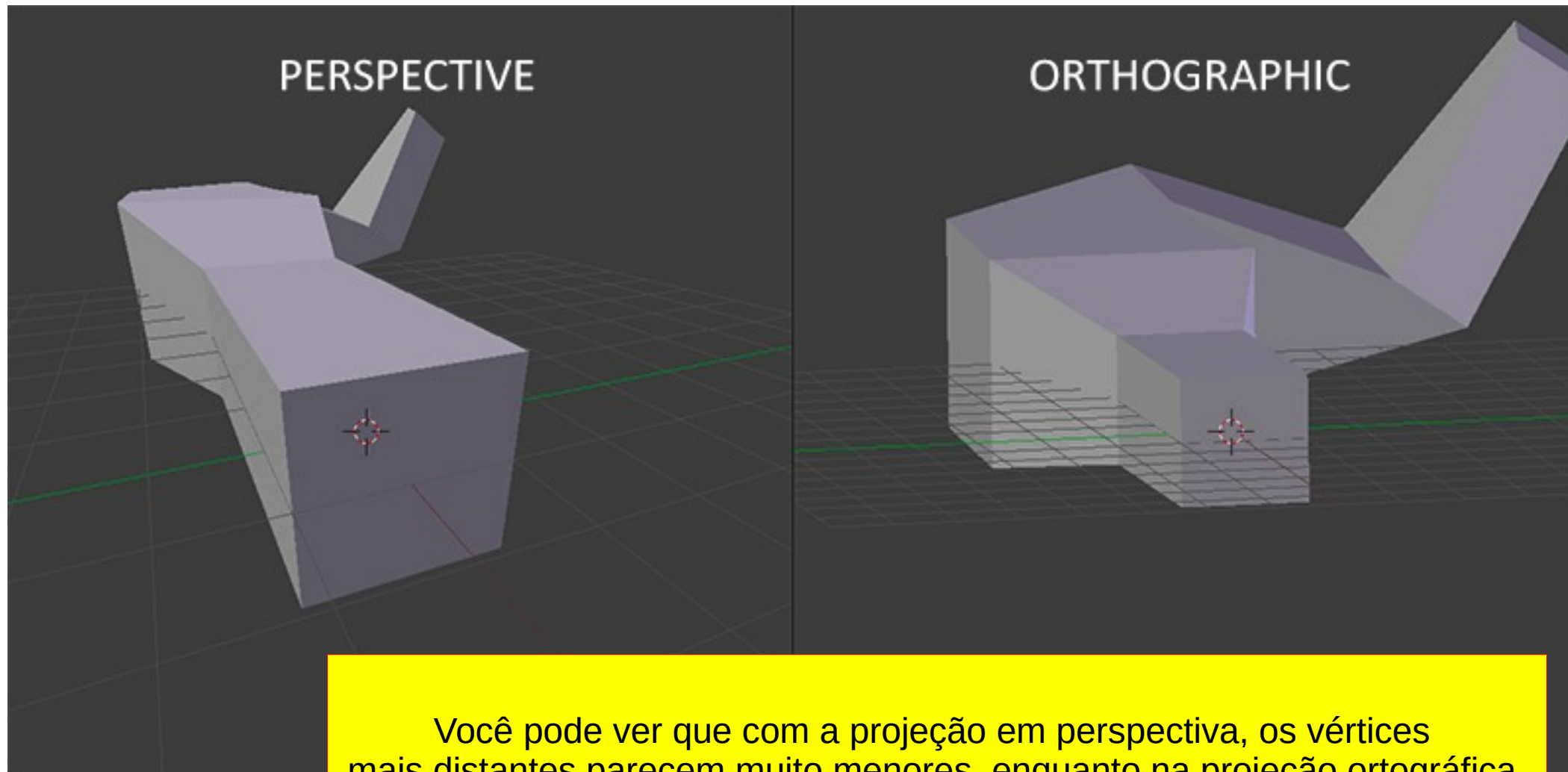
glm::perspective

- O que **glm::perspective** faz é criar novamente um grande tronco que define o espaço visível, qualquer coisa fora do tronco não terminará no volume do espaço do clipe e, portanto, será cortado.
- Um tronco de perspectiva pode ser visualizado como uma caixa de formato não uniforme de onde cada coordenada dentro desta caixa será mapeada para um ponto no espaço do clipe.

glm::perspective



Projeção Ortográfica ou de Perspectiva ?



Você pode ver que com a projeção em perspectiva, os vértices mais distantes parecem muito menores, enquanto na projeção ortográfica cada vértice tem a mesma distância para o usuário.

Projeção Ortográfica ou de Perspectiva ?

- Ao usar a **projeção ortográfica**, cada uma das coordenadas do vértice é **mapeada diretamente** para o espaço de recorte sem nenhuma divisão de perspectiva (ainda faz a divisão de perspectiva, mas o componente w não é manipulado (permanece 1) e, portanto, não tem efeito).
- Como a projeção ortográfica não usa projeção em perspectiva, os objetos mais distantes não parecem menores, o que produz uma **saída visual estranha**.
- Por esta razão, a **projeção ortográfica é usada** principalmente para renderizações 2D e **para algumas aplicações de arquitetura ou engenharia** onde prefere-se não ter vértices distorcidos pela perspectiva.
- Aplicativos como o **Blender** que são usados para modelagem 3D às vezes usam projeção ortográfica para modelagem, porque representa com mais precisão as dimensões de cada objeto.

Juntando tudo

- Criamos uma matriz de transformação para cada uma das etapas mencionadas: modelo, visualização e matriz de projeção. Uma coordenada de vértice é então transformada em coordenadas de recorte da seguinte forma:

$$V_{clip} = M_{projection} \cdot M_{view} \cdot M_{model} \cdot V_{local}$$

- Observe que a ordem da multiplicação da matriz é invertida (lembre-se de que precisamos ler a multiplicação da matriz da direita para a esquerda). O vértice resultante deve então ser atribuído a **gl_Position** no **vertex shader** e o OpenGL executará automaticamente a divisão de perspectiva e o recorte.

E depois?

- A saída do **vertex shader** requer que as coordenadas estejam no espaço de recorte, que é o que acabamos de fazer com as **matrizes de transformação**.
- O OpenGL então realiza a divisão de perspectiva nas coordenadas do espaço do clipe para transformá-las em coordenadas de dispositivo normalizadas.
- Depois, o OpenGL usa os parâmetros de **glViewport** para mapear as coordenadas do dispositivo normalizado para as coordenadas da tela, onde cada coordenada corresponde a um ponto em sua tela (em nosso caso, uma tela de 800x600).
- Esse processo é chamado de transformação da janela de visualização.

Roteiro

- GLM
- Sistemas de Coordenadas
- Espaços de Coordenadas
- Projeção Ortográfica
- Projeção em Perspectiva
- Programando em 3D

Programando em 3D

- Começa-se a desenvolver em 3D criando-se as três matrizes, **model**, **view** e **projection**, como matrizes identidade.

$$I_4 = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Uma matriz identidade possui valores iguais a 1 em sua diagonal principal e 0 nas outras posições

Definindo model, view e projection

```
glm::mat4 model = glm::mat4(1.0f);
```

```
glm::mat4 view = glm::mat4(1.0f);
```

```
glm::mat4 projection = glm::mat4(1.0f);
```

A matriz identidade pode ser criada com a função `glm::mat4(..)` da biblioteca GLM

Girando sobre o plano x

- Para começar a desenhar em 3D, primeiro definimos uma matriz de modelo. A matriz do modelo consiste em translações, escalas e / ou rotações que gostaríamos de aplicar para transformar todos os vértices do objeto para o espaço mundial global.
- Vamos transformar nosso plano um pouco girando-o no eixo x para que pareça que está no chão. A matriz do modelo é alterada da seguinte forma:

```
glm::mat4 model = glm::mat4(1.0f);
```

```
model = glm::rotate(model, glm::radians(-55.0f),  
glm::vec3(1.0f, 0.0f, 0.0f));
```



55 graus de rotação
sobre o eixo x

Girando sobre o plano x

- Para começar a desenhar em 3D, primeiro criaremos uma matriz de modelo. A matriz do modelo consiste em translações, escalas e / ou rotações que gostaríamos de aplicar para transformar todos os vértices do objeto para o espaço mundial global.
- Vamos transformar nosso plano um pouco girando-o no eixo x para que pareça que está no chão. A matriz do modelo se parece com isto:

```
glm::mat4 model = glm::mat4(1.0f);
```

```
model = glm::rotate(model, glm::radians(-55.0f),  
glm::vec3(1.0f, 0.0f, 0.0f));
```



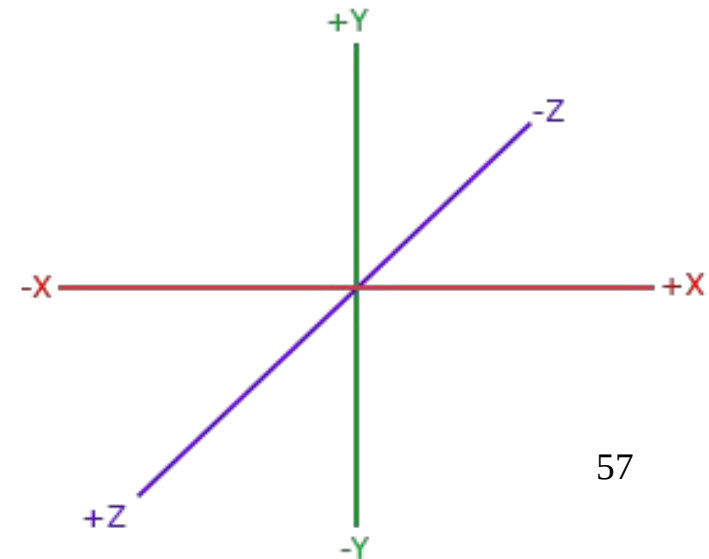
55 graus de rotação
sobre o eixo x

Ao multiplicar as coordenadas do vértice com esta matriz de modelo, estamos transformando as coordenadas do vértice em coordenadas mundiais. Nosso plano que está ligeiramente no chão representa, portanto, o plano no mundo global.

Afastando a câmera

- Em seguida, precisamos ajustar a matriz de visualização.
- Queremos nos mover um pouco para trás na cena para que o objeto se torne visível (quando no espaço do mundo estamos localizados na origem $(0,0,0)$).
- Para mover-se pela cena, pense no seguinte: Mover a câmera para trás é o mesmo que mover toda a cena para frente.

```
view = glm::translate(view, glm::vec3(0.0f, 0.0f, -3.0f));
```



Ajustando o ângulo da câmera

- A última coisa que precisamos definir é a matriz de projeção. Queremos usar a projeção em perspectiva para a nossa cena, então vamos ajustar a **matriz de projeção** assim:

```
projection = glm::perspective(glm::radians(45.0f),  
800.0f / 600.0f, 0.1f, 100.0f);
```

- Aqui são definidos:
 - Grau de abertura do ângulo de projeção : **45°**
 - Proporção da tela : **800 x 600**
 - Distância do plano próximo : **0,1**
 - Distância do plano distante : **100**

Computação Gráfica

$$q = \frac{1}{\tan\left(\frac{\text{fieldOfView}}{2}\right)}$$

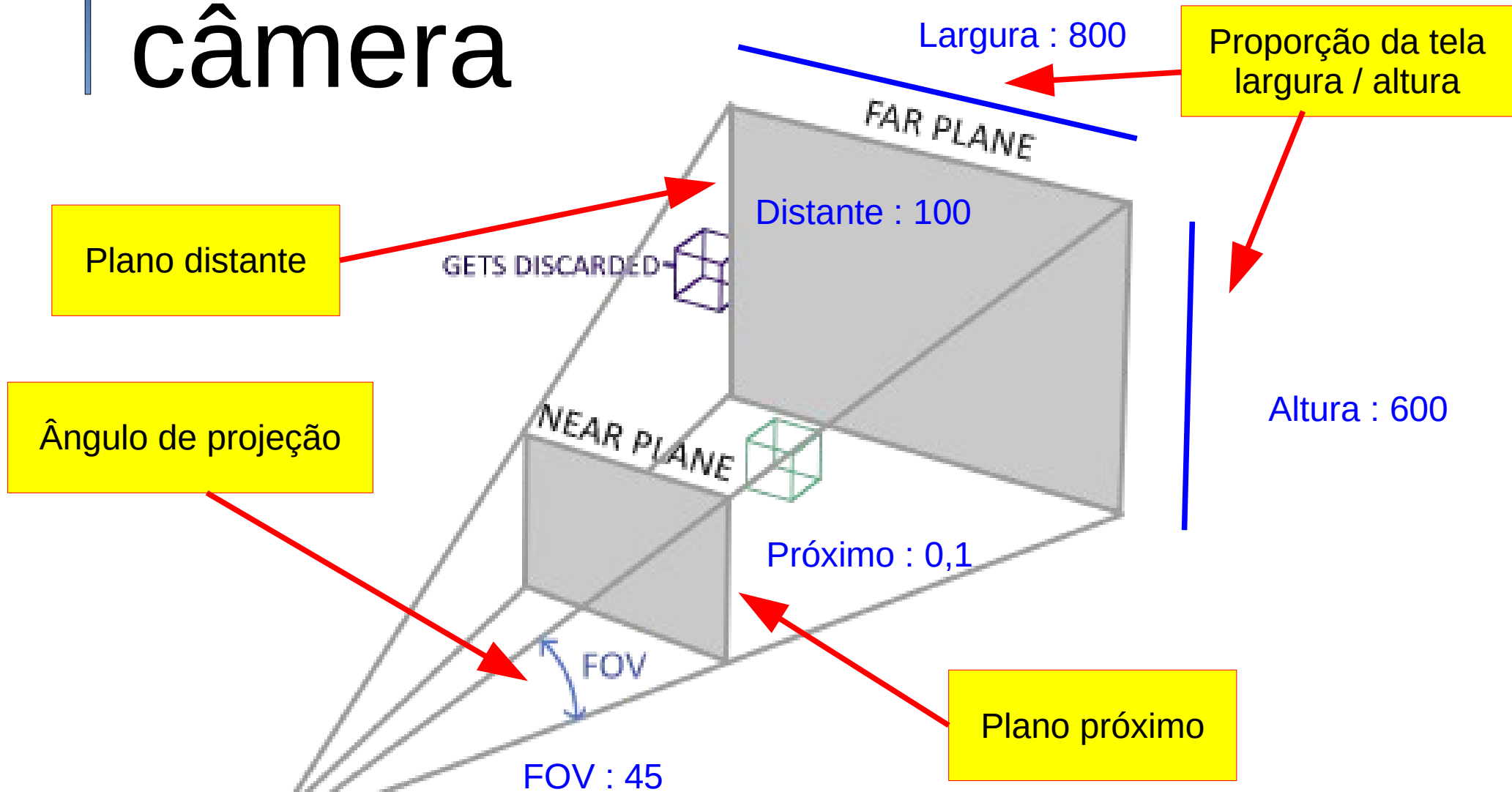
$$A = \frac{q}{\text{aspectRatio}}$$

$$B = \frac{Z_{\text{near}} + Z_{\text{far}}}{Z_{\text{near}} - Z_{\text{far}}}$$

$$C = \frac{2 * (Z_{\text{near}} * Z_{\text{far}})}{Z_{\text{near}} - Z_{\text{far}}}$$

$$\begin{bmatrix} A & 0 & 0 & 0 \\ 0 & q & 0 & 0 \\ 0 & 0 & B & C \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

Ajustando o ângulo da câmera



```
projection = glm::perspective(glm::radians(45.0f), 800.0f / 600.0f, 0.1f, 100.0f);
```

Enviar dados aos shaders

- Uma vez que as matrizes estão ajustadas, elas devem ser informadas aos **shaders**, através de **uniforms**:

Enviar dados aos shaders

```
while (!glfwWindowShouldClose(window))  
{
```

Os uniforms são localizados

```
    unsigned int modelLoc = glGetUniformLocation(ourShader.ID, "model");  
    unsigned int viewLoc  = glGetUniformLocation(ourShader.ID, "view");  
    unsigned int projLoc  = glGetUniformLocation(ourShader.ID, "projection");
```

```
    glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));  
    glUniformMatrix4fv(viewLoc, 1, GL_FALSE, &view[0][0]);  
    glUniformMatrix4fv(projLoc, 1, GL_FALSE, &projection[0][0]);
```

```
}
```

E as matrizes enviadas ao **vertex shader** através deles

Vertex shader

```
#version 330 core
layout (location = 0) in vec3 aPos;
layout (location = 1) in vec2 aTexCoord;

out vec2 TexCoord;

uniform mat4 model;
uniform mat4 view;
uniform mat4 projection;

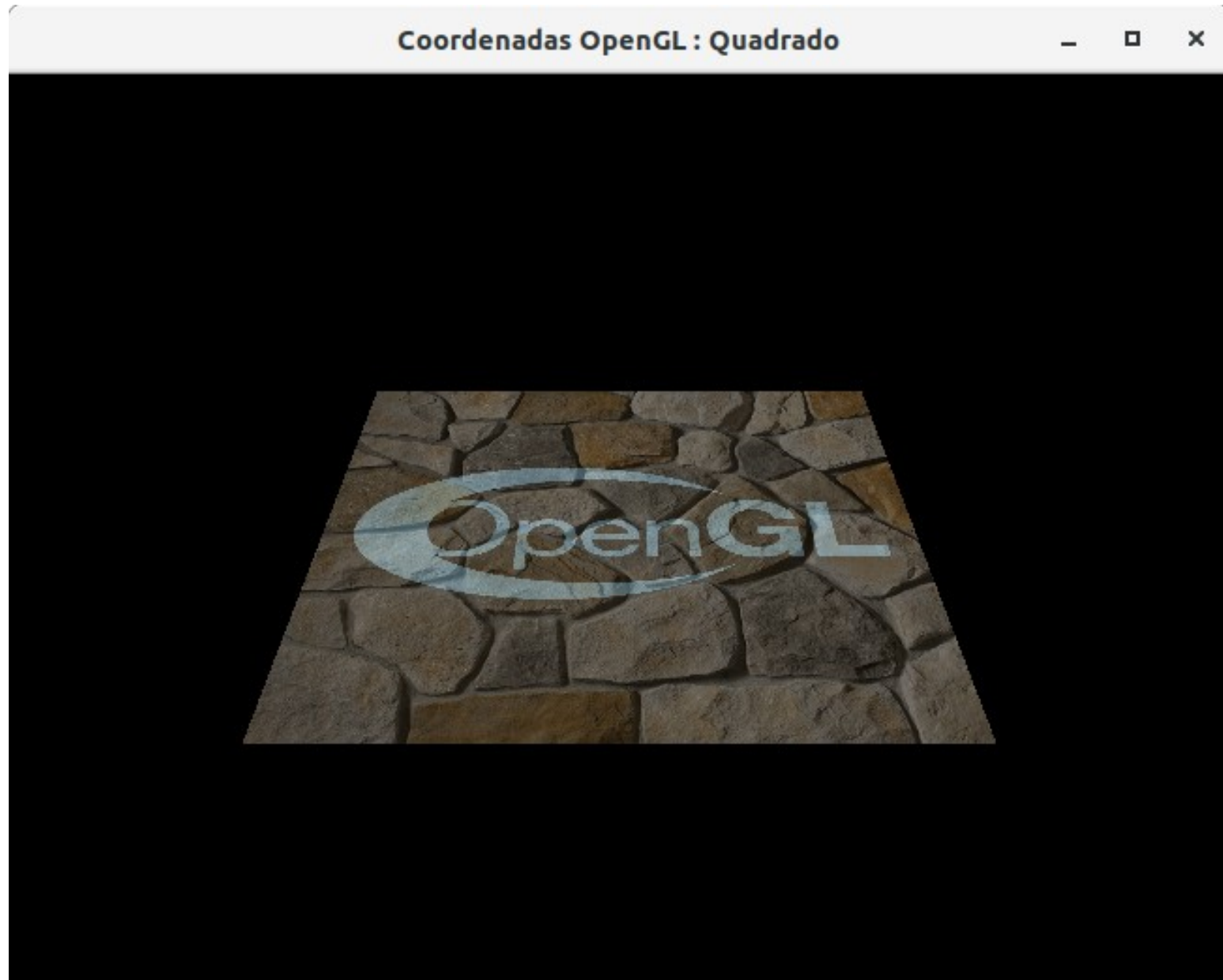
void main()
{
    gl_Position = projection * view * model * vec4(aPos, 1.0);

    TexCoord = vec2(aTexCoord.x, aTexCoord.y);
}
```

Aqui a posição de cada vértice é multiplicada pelas matrizes para gerar sua Posição final na tela



Resultado

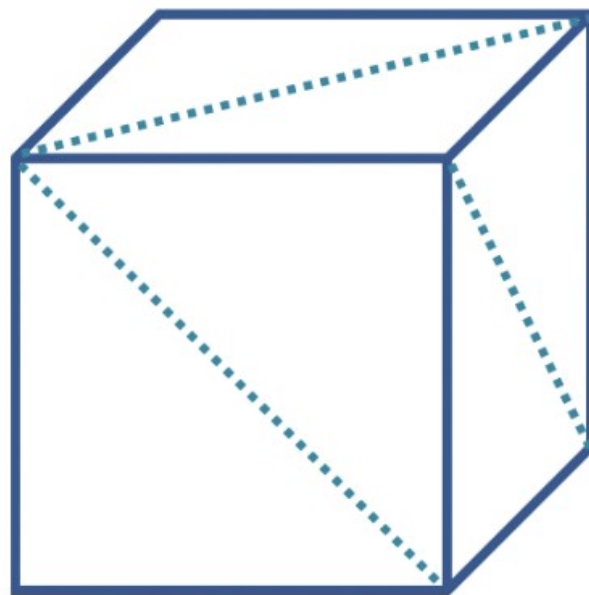


Programando em 3D

- Mas esse exemplo é apenas um plano inclinado.
- Como fazer para gerar um objeto tridimensional, como um cubo?

Desenhando um cubo

- O primeiro grande desafio em desenhar um cubo é o número de vértices.
- O cubo é formado por triângulos
 - Cada vértice tem 3 coordenadas
 - Cada superfície possui 2 triângulos
- Total : 6 faces * 2 triângulos * 3 vértices cada



```
float vertices[] = {  
    -0.5f, -0.5f, -0.5f, 0.0f, 0.0f,  
    0.5f, -0.5f, -0.5f, 1.0f, 0.0f,  
    0.5f, 0.5f, -0.5f, 1.0f, 1.0f,  
    0.5f, 0.5f, -0.5f, 1.0f, 1.0f,  
    -0.5f, 0.5f, -0.5f, 0.0f, 1.0f,  
    -0.5f, -0.5f, -0.5f, 0.0f, 0.0f,  
  
    -0.5f, -0.5f, 0.5f, 0.0f, 0.0f,  
    0.5f, -0.5f, 0.5f, 1.0f, 0.0f,  
    0.5f, 0.5f, 0.5f, 1.0f, 1.0f,  
    0.5f, 0.5f, 0.5f, 1.0f, 1.0f,  
    -0.5f, 0.5f, 0.5f, 0.0f, 1.0f,  
    -0.5f, -0.5f, 0.5f, 0.0f, 0.0f,  
  
    -0.5f, 0.5f, 0.5f, 1.0f, 0.0f,  
    -0.5f, 0.5f, -0.5f, 1.0f, 1.0f,  
    -0.5f, -0.5f, -0.5f, 0.0f, 1.0f,  
    -0.5f, -0.5f, -0.5f, 0.0f, 1.0f,  
    -0.5f, -0.5f, 0.5f, 0.0f, 0.0f,  
    -0.5f, 0.5f, 0.5f, 1.0f, 0.0f,  
  
    0.5f, 0.5f, 0.5f, 1.0f, 0.0f,  
    0.5f, 0.5f, -0.5f, 1.0f, 1.0f,  
    0.5f, -0.5f, -0.5f, 0.0f, 1.0f,  
    0.5f, -0.5f, -0.5f, 0.0f, 1.0f,  
    0.5f, -0.5f, 0.5f, 0.0f, 0.0f,  
    0.5f, 0.5f, 0.5f, 1.0f, 0.0f,  
  
    -0.5f, -0.5f, -0.5f, 0.0f, 1.0f,  
    0.5f, -0.5f, -0.5f, 1.0f, 1.0f,  
    0.5f, -0.5f, 0.5f, 1.0f, 0.0f,  
    0.5f, -0.5f, 0.5f, 1.0f, 0.0f,  
    -0.5f, -0.5f, 0.5f, 0.0f, 0.0f,  
    -0.5f, -0.5f, -0.5f, 0.0f, 1.0f,  
  
    -0.5f, 0.5f, -0.5f, 0.0f, 1.0f,  
    0.5f, 0.5f, -0.5f, 1.0f, 1.0f,  
    0.5f, 0.5f, 0.5f, 1.0f, 0.0f,  
    0.5f, 0.5f, 0.5f, 1.0f, 0.0f,  
    -0.5f, 0.5f, 0.5f, 0.0f, 0.0f,  
    -0.5f, 0.5f, -0.5f, 0.0f, 1.0f  
};
```

Desenhando um cubo

- Vamos analisar o exemplo **GLFW_Coordenadas_3D_Cubo**
- Ele é quase o mesmo exemplo que o quadrado apresentado, com algumas diferenças.
 - 1) A primeira, óbvia, é o número de vertices

Desenhando um cubo

2) Não foram definidos índices (IBO) para os vértices do cubo, e portanto o mesmo deve ser desenhado utilizando:

```
glDrawArrays(GL_TRIANGLES, 0, 36);
```

Cubo transparente?

Coordenadas OpenGL : Cubo



Com esse código, já é possível desenhar o cubo na tela, porém o mesmo terá uma aparência transparente

Transparência

- O efeito de transparência ocorre porque quando o OpenGL desenha seu cubo triângulo por triângulo, fragmento por fragmento, ele sobrescreverá qualquer cor de pixel que já tenha sido desenhada lá antes.
- Uma vez que o **OpenGL não oferece nenhuma garantia sobre a ordem dos triângulos renderizados** (dentro da mesma chamada de desenho), alguns triângulos são desenhados um em cima do outro, embora um deva estar claramente na frente do outro.
- Felizmente, o **OpenGL armazena informações de profundidade em um buffer chamado z-buffer**, que permite ao OpenGL decidir quando desenhar sobre um pixel e quando não. Usando o z-buffer, **podemos configurar o OpenGL para fazer testes de profundidade.**

Girando o cubo

Coordenadas OpenGL : Cubo



Para girar o cubo para a esquerda,
Aplica-se uma rotação negativa
sobre o eixo y

```
model = glm::rotate(model, 0.3f, glm::vec3(0.0f, -0.5f, 0.0f));
```

Girando o cubo

Coordenadas OpenGL : Cubo



Para girar o cubo para a esquerda,
Aplica-se uma rotação negativa
sobre o eixo y

Gira sobre **X**

Gira sobre **y**

Gira sobre **Z**

```
model = glm::rotate(model, 0.3f, glm::vec3(0.0f, -0.5f, 0.0f));
```

Eliminando a transparência

Coordenadas OpenGL : Cubo



Antes do loop principal, configure o OpenGL para fazer testes de profundidade, definindo:

```
glEnable(GL_DEPTH_TEST);
```

Dentro do loop, ao “limpar as cores”, execute:

```
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
```



Exemplos

- Os seguintes projetos exemplo foram disponibilizados junto com esses slides:
 - **GLFW_Coordenadas_3D_Quadrado:**
Exemplo com plano inclinado e envio de matrizes para o shader
 - **GLFW_Coordenadas_3D_Cubo :**
Exemplo de visualização do cubo tridimensional

Dúvidas?



Atividade 1

- Importe os projetos de exemplo e faça-os funcionar em seu computador

Atividade 2

- No exemplo do quadrado, como fazê-lo rodar sobre seu centro, no sentido horário?

Atividade 3

- Como alterar o exemplo do cubo para outra forma geométrica, como uma pirâmide, por exemplo?
- Faça sua pirâmide rodar sobre outros eixos
- Aumente a distância da câmera de sua pirâmide e observe os efeitos
- Aproxime a distancia da câmera de sua pirâmide e observe os efeitos